CFG防护机制简单实践与介绍

发布于 2024-12-31 20:07:29 816 0 举报

简介

Control Flow Guard（CFG）是较新的windows漏洞利用缓解措施，旨在解决内存损坏漏洞。 针对的是间接跳转的保护，比如call eax，jmp eax等。 CFG扩展了先前的漏洞缓解技术，例如GS，DEP和ASLR。

这个保护措施从Microsoft Visual Studio 2015及以上开始支持。

系统的话是从windows 8.1开始吧

简单来说就是在call eax等间接跳转之前加个验证。

实践

使用的示例代码

代码语言：javascript AI代码解释

```
typedef int(*fun_t)(int);

int foo(int a)
{
    printf("hellow world %d\n",a);
    return a;
}
class CTargetObject
{
public:
    fun_t fun;
};
int main()
{
    int i = 0;
    CTargetObject *o_array = new CTargetObject[5];
    for (i = 0; i < 5 ; i++)
        o_array[i].fun = foo;
    o_array[0].fun(1);
    return 0;
}
```

修改编译选项

learnCFG 属性页

配置(C): 活动(Debug) 平台(P): 活动(Win32) 配置管理器(O)...

- 配置属性
  - 常规
  - 调试
  - VC++ 目录
  - C/C++
    - 常规
    - 优化
    - 预处理器
    - 代码生成
    - 语言
    - 预编译头
    - 输出文件
    - 浏览信息
    - 高级
    - 所有选项
    - 命令行
  - 链接器
  - 清单工具
  - XML 文档生成器
  - 浏览信息
  - 生成事件
  - 自定义生成步骤
  - 代码分析

启用字符串池	
启用最小重新生成	是 (/Gm)
启用 C++ 异常	是 (/EHsc)
较小类型检查	否
基本运行时检查	两者(/RTC1，等同于 /RTCsu) (/RTC1)
运行库	多线程调试 DLL (/MDd)
结构成员对齐	默认设置
安全检查	启用安全检查 (/GS)
控制流防护	
启用函数级链接	是 (/guard:cf)
启用并行代码生成	否
启用增强指令集	未设置
浮点模型	精度 (/fp:precise)
启用浮点异常	
创建可热修补映像	

控制流防护

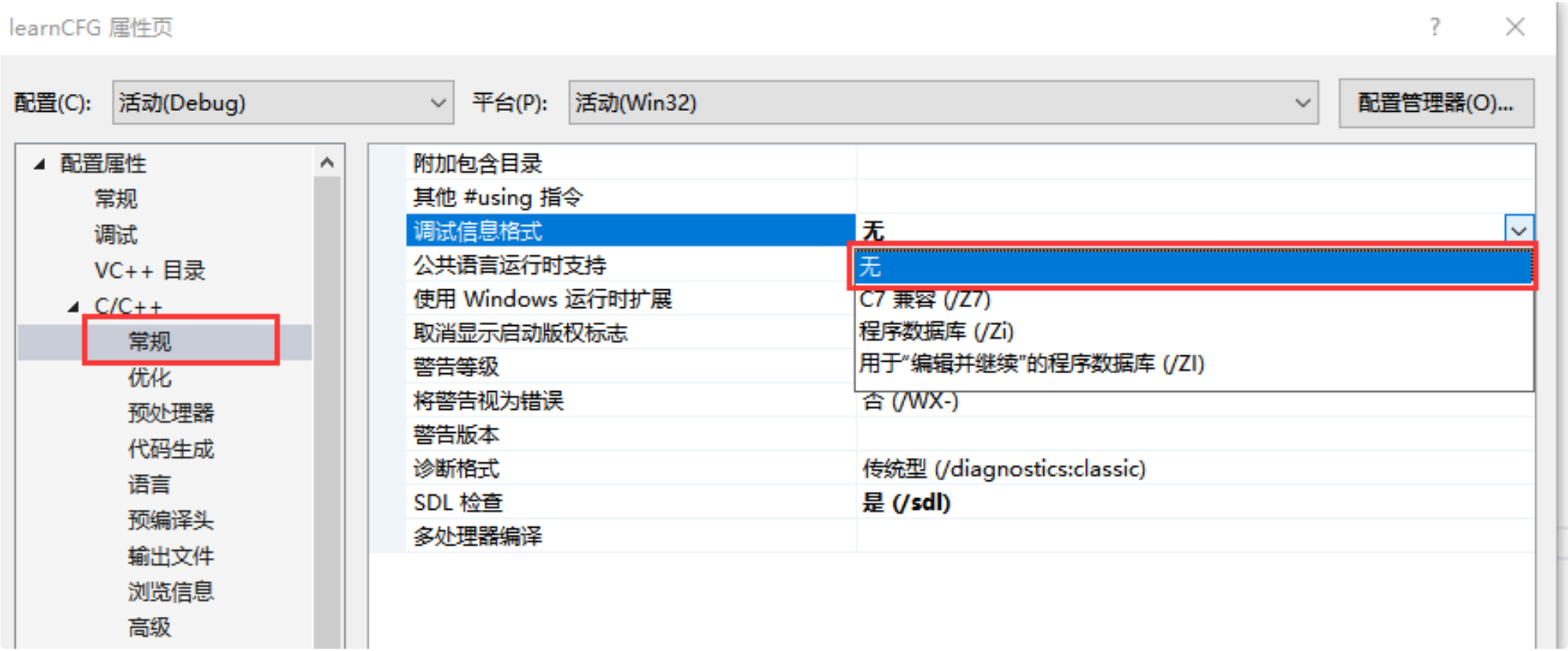
防护安全检查可帮助检测到尝试调度到非法代码块的操作。(/guard:cf)

确定 取消 应用(A)

上面改了可能报错：

命令行 error D8016: “/ZI”和“/guard:cf”命令行选项不兼容

就是CFG与这个ZI不兼容，所以我们得关闭ZI



编译完成我们可以使用winchecksec工具查看是否开启了CFG

https://github.com/trailofbits/winchecksec

```
E:\VS2017\learnCFG\winchecksec-1.1.1>winchecksec.exe ../Debug/learnCFG.exe
Dynamic Base     : true
ASLR             : true
High Entropy VA  : false
Force Integrity  : false
Isolation        : true
NX               : true
SEH              : true
CFG              : true
RFG              : false
SafeSEH          : false
GS               : false
Authenticode     : false
.NET             : true
```

或者使用VS自带的工具dumpbin.exe，命令如下：

代码语言：javascript

AI代码解释

```javascript
dumpbin.exe /headers /loadconfig E:\VS2017\learnCFG\Debug\learnCFG.exe
```

开启了CFG，OPTIONAL HEADER VALUES里面应该有Control Flow Guard

Section contains the following load config那里有“CF Instrumented”和“FID table present”等

```
OPTIONAL HEADER VALUES
             10B magic # (PE32)
           14.11 linker version
            6200 size of code
            4A00 size of initialized data
               0 size of uninitialized data
            1100 entry point (00401100) @ILT+240(_mainCRTStartup)
            1000 base of code
            8000 base of data
          400000 image base (00400000 to 00410FFF)
            1000 section alignment
             200 file alignment
            6.00 operating system version
            0.00 image version
            6.00 subsystem version
               0 Win32 version
           11000 size of image
             400 size of headers
               0 checksum
               3 subsystem (Windows CUI)
            C140 DLL characteristics
                   Dynamic base
                   NX compatible
                   Control Flow Guard
                   Terminal Server Aware
          100000 size of stack reserve
            1000 size of stack commit
```

```
Section contains the following load config:

        00000098 size
               0 time date stamp
            0.00 Version
               0 GlobalFlags Clear
               0 GlobalFlags Set
               0 Critical Section Default Timeout
               0 Decommit Free Block Threshold
               0 Decommit Total Free Threshold
        00000000 Lock Prefix Table
               0 Maximum Allocation Size
               0 Virtual Memory Threshold
               0 Process Heap Flags
               0 Process Affinity Mask
               0 CSD Version
            0000 Dependent Load Flag
        00000000 Edit List
        0040B000 Security Cookie
        00409A20 Safe Exception Handler Table
               1 Safe Exception Handler Count
        0040E000 Guard CF address of check-function pointer
        00000000 Guard CF address of dispatch-function pointer
        0040D000 Guard CF function table
              1C Guard CF function count
        00010500 Guard Flags
                   CF instrumented
                   FID table present
                   Long jump target table present
```

```
                Long jump target table present
        0000 Code Integrity Flags
        0000 Code Integrity Catalog
    00000000 Code Integrity Catalog Offset
    00000000 Code Integrity Reserved
    00000000 Guard CF address taken IAT entry table
           0 Guard CF address taken IAT entry count
    00000000 Guard CF long jump target table
```

我们看看开了CFG和没有开的区别

首先是没开的

```
.text:004119D0                 cmp     [ebp+var_8], 5
.text:004119D4                 jge     short loc_4119E5
.text:004119D6 ; 9:     v2[i] = (void (__cdecl *)(signed int))sub_4111B3;
.text:004119D6                 mov     eax, [ebp+var_8]
.text:004119D9                 mov     ecx, [ebp+var_14]
.text:004119DC                 mov     dword ptr [ecx+eax*4], offset sub_4111B3
.text:004119E3                 jmp     short loc_4119C7
.text:004119E5 ; ---------------------------------------------------------------------------
.text:004119E5 ; 10:    (*v2)(1);
.text:004119E5
.text:004119E5 loc_4119E5:                             ; CODE XREF: _main_0+54↑j
.text:004119E5                 mov     esi, esp
.text:004119E7                 push    1
.text:004119E9                 mov     eax, 4
.text:004119EE                 imul    ecx, eax, 0
.text:004119F1                 mov     edx, [ebp+var_14]
.text:004119F4                 mov     eax, [edx+ecx]
.text:004119F7                 call    eax
.text:004119F9 ; 11:    return 0;
.text:004119F9                 add     esp, 4
.text:004119FC                 cmp     esi, esp
.text:004119FE                 call    j___RTC_CheckEsp
.text:00411A03                 xor     eax, eax
.text:00411A05                 pop     edi
```

没有CFG

接下来是开了的

```
.text:004021B7 ; 15:    v1 = 1;
.text:004021B7
.text:004021B7 loc_4021B7:                                ; CODE XREF: _main_0+56↑j
.text:004021B7                 mov     esi, esp
.text:004021B9                 push    1
.text:004021BB ; 16:    v2 = *v4;
.text:004021BB                 mov     ecx, 4
.text:004021C0                 imul    edx, ecx, 0
.text:004021C3                 mov     eax, [ebp+var_C]
.text:004021C6                 mov     ecx, [eax+edx]
.text:004021C9                 mov     [ebp+var_14], ecx
.text:004021CC ; 17:    v2(1);
.text:004021CC                 mov     edi, esp
.text:004021CE                 mov     ecx, [ebp+var_14]
.text:004021D1                 call    ds:___guard_check_icall_fptr ; cfg检查
.text:004021D7                 cmp     edi, esp
.text:004021D9                 call    j___RTC_CheckEsp
.text:004021DE                 call    [ebp+var_14]    ; 这个来源ecx，相当于call ecx
.text:004021E1 ; 18:    return 0;
.text:004021E1                 add     esp, 4
.text:004021E4                 cmp     esi, esp
```

双击跟过去那个检查函数是没有代码的，这应该是运行程序的时候再填充了

```
.00cfg:0040E000                 assume cs:_00cfg
.00cfg:0040E000                 ;org 40E000h
.00cfg:0040E000 ___guard_check_icall_fptr dd offset j_nullsub_1
.00cfg:0040E000                                         ; DATA XREF: _main_0+81↑r
.00cfg:0040E000                                         ; __guard_icall_checks_enforced↑r ...
.00cfg:0040E004                 align 1000h
.00cfg:0040E004 _00cfg          ends
.00cfg:0040E004
.00cfg:0040E004
.00cfg:0040E004                 end start
```

运行时，实际调用的是下面的ntdll!LdrpValidateUserCallTarget

```
007121cc 8bfc          mov     edi,esp
007121ce 8b4dec        mov     ecx,dword ptr [ebp-14h]
007121d1 ff1500e07100  call    dword ptr [learnCFG!__guard_check_icall_fptr (0071e000)] ds:002b:0071e000={ntdll!LdrpValidateUserCallTarget (77408be0)}
007121d7 3bfc          cmp     edi,esp
007121d9 e802f1ffff    call    learnCFG!ILT+720(__RTC_CheckEsp) (007112e0)
007121de ff55ec        call    dword ptr [ebp-14h]
007121e1 83c404        add     esp,4
007121e4 3bf4          cmp     esi,esp
007121e6 e8f5f0ffff    call    learnCFG!ILT+720(__RTC_CheckEsp) (007112e0)
007121eb 33c0          xor     eax,eax
007121ed 5f            pop     edi
007121ee 5e            pop     esi
007121ef 8b4dfc        mov     ecx,dword ptr [ebp-4]
007121f2 33cd          xor     ecx,ebp
007121f4 e847f4ffff    call    learnCFG!ILT+1584(__security_check_cookie (00711640)
007121f9 83c414        add     esp,14h
007121fc 3bec          cmp     ebp,esp
007121fe e8ddf0ffff    call    learnCFG!ILT+720(__RTC_CheckEsp) (007112e0)
```

```
Command
ntdll!LdrpDbgBreakpoint+0x2b:
7742e9d2 cc              int     3
0:000> bm
        ^ Non-empty string required in 'bm'
0:000> lm
start    end        module name
00710000 00721000   learnCFG   (deferred)
76570000 7676c000   KERNELBASE   (deferred)
76e90000 76f70000   KERNEL32   (deferred)
77380000 7751a000   ntdll      (pdb symbols)          C:\Program Files (x86)\Windows Kits\10\Debuggers\x86\sym\wntdll.pdb\D85FCE08D56038E2C69B69F29E11B5EE1`
79ea0000 79eba000   VCRUNTIME140D   (deferred)
79ec0000 7a031000   ucrtbased   (deferred)
0:000> bp 007121D1
*** WARNING: Unable to verify checksum for learnCFG.exe
0:000> g
Breakpoint 0 hit
eax=00446db8 ebx=002f5000 ecx=007113a0 edx=00000000 esi=0019f740 edi=0019f73c
eip=007121d1 esp=0019f73c ebp=0019f75c iopl=0         nv up ei pl nz na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00000206
learnCFG!main+0x81:
007121d1 ff1500e07100    call    dword ptr [learnCFG!__guard_check_icall_fptr (0071e000)] ds:002b:0071e000={ntdll!LdrpValidateUserCallTarget (77408be0)}
0:000> dd 0071e000
0071e000  77408be0 00000000 00000000 00000000
0071e010  00000000 00000000 00000000 00000000
0071e020  00000000 00000000 00000000 00000000
0071e030  00000000 00000000 00000000 00000000
0071e040  00000000 00000000 00000000 00000000
0071e050  00000000 00000000 00000000 00000000
0071e060  00000000 00000000 00000000 00000000
0071e070  00000000 00000000 00000000 00000000
0:000>
```

原理分析

首先说说CFGBitmap

pCFG检查基于CFGBitmap，它表示在进程空间内所有函数的起始位置。在进程空间内每8个字节的状态对应CFGBitmap中的一位。如果函数的地址是合法有效的，那么这个函数对应在CFGbitmap的位置会被设置位1，否则是0。 p p一个Bitmap的大小是4字节

下面以地址0x00b01030为例（这是先知参考文章的例子）

首先获取对应的CFGBitmap的地址

CFGBitmap基址+偏移 = CFGBitmap基址 + （要检测的地址的高3个字节）* 4 = CFGBitmap基址 + 0x00b010*4

乘以4，因为Bitmap的大小是4字节

假如取到的bitmap如图 00000100 00000000 00000000 01000000

00000000 10110000 00010000 00110000

0x00b01030二进制是上图，取低字节的高5位，记为X

接着判断目标地址的最低字节是否以0x10对齐： 即0x30 & 0xf 是否等于 0，结果是等于0的

- 如果等于0，X就是偏移,即 00110，也即十进制的6
- 否则偏移是 X or 0x1 （这是趋势的文章，旧版的好像是这样，可查看先知的文章，我调试的实际情况是将X最低位置0，作为索引）

上面算出来的6就是bitmap的偏移（偏移从0算起），那么下面偏移6的位置为1，即函数是有效函数，之后检测到是0还会再进一步检查（将X or 1作为索引再检查）

00000100 00000000 00000000 01000000

具体调试结果跟解析如下了：

代码语言：javascript

AI代码解释

```
ntdll!LdrpValidateUserCallTarget:
77408be0 8b15f8124a77    mov     edx,dword ptr [ntdll!LdrSystemDllInitBlock+0xb0 (774a12f8)] ds:002b:774a12f8=00730000
;获取CFGBitmap基址
77408be6 8bc1            mov     eax,ecx    ;ecx为要检验的函数地址，给到eax (ecx=007113a0)
77408be8 c1e808          shr     eax,8      ;取地址的高3个字节 (eax = 00007113)
ntdll!LdrpValidateUserCallTargetBitMapCheck:
77408beb 8b1482          mov     edx,dword ptr [edx+eax*4] ds:002b:0074c44c=10100444 ;获取该地址对应的bitmap
77408bee 8bc1            mov     eax,ecx
77408bf0 c1e803          shr     eax,3                          ;舍弃最低3个bit
77408bf3 f6c10f          test    cl,0Fh                         ;判断目标地址是否以0x10对齐，跟0xf与运算，判断是否等于 0
77408bf6 7506            jne     ntdll!LdrpValidateUserCallTargetBitMapRet+0x1 (77408bfe)   ;不等于0则跳转到77408bfe
; bt系列指令它们的结果影响CF标志位
;BT:把指定位传送给CF;
;BTC:把指定位传送给CF后还使该位变反;
;BTR:把指定位传送给CF后还使该位变为0;
;BTS:把指定位传送给CF后还使该位变为1;
77408bf8 0fa3c2          bt      edx,eax                        ;将edx的第eax位，给到CF（其实bt指令最多取eax的低5个bit）
;假如bt的第二个参数很大，32位取低3bit，64位取低5bit，参考http://faydoc.tripod.com/cpu/bt.htm的Description的第二段
;edx为00010000 00010000 00000100 01000100，eax的最低5bit为10100，即20，可以看到索引20位置为1，置CF为1，所以是有效的
77408bfb 730a            jae     ntdll!LdrpValidateUserCallTargetBitMapRet+0xa (77408c07) ;CF等于0才跳
ntdll!LdrpValidateUserCallTargetBitMapRet:
77408bfd c3              ret                            ;正常情况下在这就返回执行正常代码了
77408bfe 0fbaf000        btr     eax,0                          ;跟0x10不对齐跳到这，将eax的最低bit给到CF，之后将最低bit置0
77408c02 0fa3c2          bt      edx,eax                        ;跟上面的一样，取低5bit作为edx的bit的索引，给到CF
77408c05 7309            jae     ntdll!LdrpValidateUserCallTargetBitMapRet+0x13 (77408c10) ;这就跳到失败的流程了
77408c07 83c801          or      eax,1                          ;上面77408bfb判断无效后，来到这将eax最低位置1后再判断
77408c0a 0fa3c2          bt      edx,eax
77408c0d 7301            jae     ntdll!LdrpValidateUserCallTargetBitMapRet+0x13 (77408c10) ;这就跳到失败的流程了
77408c0f c3              ret
77408c10 51              push    ecx
77408c11 8d642480        lea     esp,[esp-80h]
77408c15 0f110424        movups  xmmword ptr [esp],xmm0
77408c19 0f114c2410      movups  xmmword ptr [esp+10h],xmm1
77408c1e 0f11542420      movups  xmmword ptr [esp+20h],xmm2
77408c23 0f115c2430      movups  xmmword ptr [esp+30h],xmm3
77408c28 0f11642440      movups  xmmword ptr [esp+40h],xmm4
77408c2d 0f116c2450      movups  xmmword ptr [esp+50h],xmm5
77408c32 0f11742460      movups  xmmword ptr [esp+60h],xmm6
```

```
77408c37 0f117c2470      movups  xmmword ptr [esp+70h],xmm7
77408c3c e8e7460500      call    ntdll!RtlpHandleInvalidUserCallTarget (7745d328)
77408c41 0f100424        movups  xmm0,xmmword ptr [esp]
77408c45 0f104c2410      movups  xmm1,xmmword ptr [esp+10h]
77408c4a 0f10542420      movups  xmm2,xmmword ptr [esp+20h]
77408c4f 0f105c2430      movups  xmm3,xmmword ptr [esp+30h]
77408c54 0f10642440      movups  xmm4,xmmword ptr [esp+40h]
77408c59 0f106c2450      movups  xmm5,xmmword ptr [esp+50h]
77408c5e 0f10742460      movups  xmm6,xmmword ptr [esp+60h]
77408c63 0f107c2470      movups  xmm7,xmmword ptr [esp+70h]
77408c68 8da42480000000  lea     esp,[esp+80h]
77408c6f 59              pop     ecx
77408c70 c3              ret
```

那么最终假如校验失败，那就会跳到

代码语言：javascript

AI代码解释

```
77408c3c e8e7460500      call    ntdll!RtlpHandleInvalidUserCallTarget (7745d328)
```

```
ntdll!LdrpValidateUserCallTargetBitMapCheck:
77408beb 8b1482          mov     edx,dword ptr [edx+eax*4]
77408bee 8bc1            mov     eax,ecx
77408bf0 c1e803          shr     eax,3
77408bf3 f6c10f          test    cl,0Fh
77408bf6 7506            jne     ntdll!LdrpValidateUserCallTargetBitMapRet+0x1 (77408bfe)
77408bf8 0fa3c2          bt      edx,eax
77408bfb 730a            jae     ntdll!LdrpValidateUserCallTargetBitMapRet+0xa (77408c07)
ntdll!LdrpValidateUserCallTargetBitMapRet:
77408bfd c3              ret
77408bfe 0fbaf000        btr     eax,0
77408c02 0fa3c2          bt      edx,eax
77408c05 7309            jae     ntdll!LdrpValidateUserCallTargetBitMapRet+0x13 (77408c10)
77408c07 83c801          or      eax,1
77408c0a 0fa3c2          bt      edx,eax
77408c0d 7301            jae     ntdll!LdrpValidateUserCallTargetBitMapRet+0x13 (77408c10) [br=1]
77408c0f c3              ret
77408c10 51              push    ecx
77408c11 8d642480        lea     esp,[esp-80h]
77408c15 0f110424        movups  xmmword ptr [esp],xmm0
77408c19 0f114c2410      movups  xmmword ptr [esp+10h],xmm1
77408c1e 0f11542420      movups  xmmword ptr [esp+20h],xmm2
77408c23 0f115c2430      movups  xmmword ptr [esp+30h],xmm3
77408c28 0f11642440      movups  xmmword ptr [esp+40h],xmm4
77408c2d 0f116c2450      movups  xmmword ptr [esp+50h],xmm5
77408c32 0f11742460      movups  xmmword ptr [esp+60h],xmm6
77408c37 0f117c2470      movups  xmmword ptr [esp+70h],xmm7
77408c3c e8e7460500      call    ntdll!RtlpHandleInvalidUserCallTarget (7745d328)
77408c41 0f100424        movups  xmm0,xmmword ptr [esp]
77408c45 0f104c2410      movups  xmm1,xmmword ptr [esp+10h]
77408c4a 0f10542420      movups  xmm2,xmmword ptr [esp+20h]
```

校验是Invalid

最终int 0x29抛出异常

```
eax=00000000 ebx=02869000 ecx=0000000a edx=00711320 esi=00711320 edi=02aff854
eip=7745d3b6 esp=02aff79c ebp=02aff7c4 iopl=0         nv up ei pl zr na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00000246
ntdll!RtlpHandleInvalidUserCallTarget+0x8e:
7745d3b6 e865b9faff      call    ntdll!RtlFailFast2 (77408d20)
0:000> t
eax=00000000 ebx=02869000 ecx=0000000a edx=00711320 esi=00711320 edi=02aff854
eip=77408d20 esp=02aff798 ebp=02aff7c4 iopl=0         nv up ei pl zr na pe nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00000246
ntdll!RtlFailFast2:
77408d20 cd29            int     29h
0:000> kv
 # ChildEBP RetAddr  Args to Child
00 02aff794 7745d3bb 02aff858 00000038 02b16d98 ntdll!RtlFailFast2 (FPO: [0,0,0])
01 02aff7c4 77408c41 0075006c 0065006d 005c0034 ntdll!RtlpHandleInvalidUserCallTarget+0x93 (FPO: [Non-Fpo])
02 02aff84c 007121d7 00000001 00711100 00711100 ntdll!LdrpValidateUserCallTargetBitMapRet+0x44 (FPO: [0,0,0])
03 02aff874 00712a2e 00000001 02b15290 02b15d40 learnCFG!main+0x87
04 02aff888 007128a0 3a48fdcc 00711100 00711100 learnCFG!invoke_main+0x1e (FPO: [Non-Fpo]) (CONV: cdecl) [f:\dd\vctools\crt\vcstartup\src\startup\exe
05 02aff8e0 0071273d 02aff8f0 00712aa8 02aff900 learnCFG!__scrt_common_main_seh+0x150 (FPO: [Non-Fpo]) (CONV: cdecl) [f:\dd\vctools\crt\vcstartup\src\
06 02aff8e8 00712aa8 02aff900 76ea6359 02869000 learnCFG!__scrt_common_main+0xd (FPO: [Non-Fpo]) (CONV: cdecl) [f:\dd\vctools\crt\vcstartup\src\startu
07 02aff8f0 76ea6359 02869000 76ea6340 02aff95c learnCFG!mainCRTStartup+0x8 (FPO: [Non-Fpo]) (CONV: cdecl) [f:\dd\vctools\crt\vcstartup\src\startup\ex
08 02aff900 773e7b74 02869000 36bd35fd 00000000 KERNEL32!BaseThreadInitThunk+0x19 (FPO: [Non-Fpo])
09 02aff95c 773e7b44 ffffffff 77408f2e 00000000 ntdll!__RtlUserThreadStart+0x2f (FPO: [SEH])
0a 02aff96c 00000000 00711100 02869000 00000000 ntdll!_RtlUserThreadStart+0x1b (FPO: [Non-Fpo])
```

最终使用int 0x29让程序快速跑出异常

参考

https://blog.trendmicro.com/trendlabs-security-intelligence/exploring-control-flow-guard-in-windows-10/

https://docs.microsoft.com/en-us/windows/win32/secbp/control-flow-guard

https://xz.aliyun.com/t/2587

https://docs.microsoft.com/en-us/windows/win32/secbp/control-flow-guard#how-do-i-tell-that-a-binary-is-under-control-flow-guard

本文参与 腾讯云自媒体同步曝光计划，分享自作者个人站点/博客。

原始发表：2019-10-28，如有侵权请联系 cloudcommunity@tencent.com 删除

前往查看

漏洞 命令行 实践 索引 函数